DATABASE MANAGEMENT SYSTEMS

Complete Bible — 2026

Beginner · Intermediate · Advanced · Pro/Expert

SQL · Normalization · Transactions · Indexing · Query Optimization

NoSQL · Distributed DBs · Replication · Sharding · NewSQL

WHAT'S INSIDE:

40 fully-detailed chapters across 4 mastery levels

Complete SQL examples, ER diagrams, and real-world schemas

PostgreSQL, MySQL, MongoDB, Redis, Cassandra case studies

Generated: 2026

CONTENTS AT A GLANCE

QUICK REFERENCE

PART I

BEGINNER

For those starting from zero — no prior database knowledge assumed

Chapter 1

What Is a Database?

Why Not Just Use Files?

Before databases, programs stored data in flat files (CSV, text). Problems: (1) **Redundancy** — same data duplicated across files. (2) **Inconsistency** — update one copy, forget another. (3) **No concurrent access** — two users editing the same file corrupts it. (4) **No security** — no way to restrict who sees what. (5) **No relationships** — linking data across files requires custom code. Databases solve all of these.

What Is a DBMS?

A **Database Management System (DBMS)** is software that creates, manages, and provides access to databases. It sits between applications and data, providing: data definition (structure), data manipulation (CRUD), concurrency control, security, backup/recovery, and query optimization. Examples: PostgreSQL, MySQL, Oracle, SQL Server, MongoDB, Redis.

Types of Databases

Type	Data Model	Examples	Best For
Relational (RDBMS)	Tables with rows and columns	PostgreSQL, MySQL, Oracle	Structured data, ACID transactions
Document	JSON/BSON documents	MongoDB, CouchDB	Semi-structured, flexible schemas
Key-Value	Simple key → value pairs	Redis, DynamoDB, etcd	Caching, sessions, config
Column-Family	Columns grouped into families	Cassandra, HBase, ScyllaDB	Wide rows, time-series, IoT
Graph	Nodes and edges (relationships)	Neo4j, Amazon Neptune	Social networks, recommendations
Time-Series	Timestamped data points	InfluxDB, TimescaleDB	Monitoring, IoT, financial data
Vector	High-dimensional embeddings	Pinecone, Weaviate, pgvector	AI/ML similarity search

DBMS Architecture

A typical DBMS has three layers: (1) **External level** — user views (what each user sees). (2) **Conceptual level** — logical schema (tables, relationships). (3) **Internal level** — physical storage (files, indexes on disk). This **three-schema architecture** provides **data independence**: you can change storage (physical) without changing the logical schema, and change the logical schema without changing application views.

Chapter 2

Relational Model Fundamentals

The Relational Model

Proposed by Edgar F. Codd in 1970. Data is organized into **relations** (tables). Each table has **attributes** (columns) and **tuples** (rows). Each attribute has a **domain** (set of allowed values). The relational model is based on set theory and first-order predicate logic.

Key Terminology

Term	Formal Name	Everyday Name	Example
Relation	Relation	Table	students
Tuple	Tuple	Row / Record	(101, 'Alice', 3.8)
Attribute	Attribute	Column / Field	student_name
Domain	Domain	Data type / allowed values	GPA: 0.0 to 4.0
Degree	Degree	Number of columns	3 (id, name, gpa)
Cardinality	Cardinality	Number of rows	500 students

Relational Algebra

Relational algebra is the theoretical foundation for SQL. Core operations:

- **Selection (σ)**: Filter rows by condition. σ(gpa > 3.5)(students)
- **Projection (π)**: Select specific columns. π(name, gpa)(students)
- **Join (■)**: Combine rows from two tables on a condition. students ■ enrollments
- **Union (∪)**: Combine rows from two compatible tables.
- **Difference (–)**: Rows in A but not in B.
- **Cartesian Product (×)**: Every combination of rows.

Chapter 3

SQL Basics: SELECT, INSERT, UPDATE, DELETE

SQL Categories

Category	Full Name	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Define/modify schema
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Read/write data
DCL	Data Control Language	GRANT, REVOKE	Permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Transaction management

Creating Tables

```
CREATE TABLE students (
    student_id   SERIAL PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    email        VARCHAR(255) UNIQUE,
    gpa          DECIMAL(3,2) CHECK (gpa >= 0 AND gpa <= 4),
    enrolled_at  DATE DEFAULT CURRENT_DATE
);
```

CRUD Operations

```
-- INSERT: Add a row
INSERT INTO students (name, email, gpa)
VALUES ('Alice', 'alice@uni.edu', 3.85);
-- SELECT: Read data
SELECT name, gpa FROM students WHERE gpa > 3.5;
-- UPDATE: Modify existing rows
UPDATE students SET gpa = 3.90 WHERE student_id = 1;
-- DELETE: Remove rows
DELETE FROM students WHERE student_id = 42;
```

Chapter 4

Filtering, Sorting, and Aggregation

WHERE Clause Operators

```
-- Comparison: =, <>, <, >, <=, >=
SELECT * FROM orders WHERE total > 100;
-- Logical: AND, OR, NOT
SELECT * FROM products WHERE price > 10 AND category = 'Books';
-- Pattern matching: LIKE, ILIKE
SELECT * FROM customers WHERE name LIKE 'A%';  -- starts with A
-- Range: BETWEEN, IN
SELECT * FROM orders WHERE total BETWEEN 50 AND 200;
SELECT * FROM products WHERE category IN ('Books', 'Electronics');
-- NULL checks: IS NULL, IS NOT NULL
SELECT * FROM customers WHERE phone IS NULL;
```

ORDER BY and LIMIT

```
-- Sort results
SELECT name, gpa FROM students ORDER BY gpa DESC;
-- Pagination: top 10 results
SELECT * FROM products ORDER BY price ASC LIMIT 10 OFFSET 20;
```

Aggregate Functions

```
-- COUNT, SUM, AVG, MIN, MAX
SELECT COUNT(*) FROM students;                -- total rows
SELECT AVG(gpa) FROM students;                -- average GPA
SELECT department, COUNT(*) as num_students
FROM students
GROUP BY department
HAVING COUNT(*) > 10                          -- filter groups
ORDER BY num_students DESC;
```

Execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Chapter 5

JOINs — Combining Tables

JOIN Types

JOIN	Returns	Use Case
INNER JOIN	Only matching rows in both tables	Most common — get related data
LEFT JOIN	All left rows + matching right (NULL if no match)	Include all from left even if no match
RIGHT JOIN	All right rows + matching left	Rarely used (swap tables + LEFT JOIN)
FULL OUTER JOIN	All rows from both (NULL where no match)	Find unmatched records on both sides
CROSS JOIN	Every combination (Cartesian product)	Generate combinations, test data

```
-- INNER JOIN: students with their enrollments
SELECT s.name, c.course_name, e.grade
FROM students s
INNER JOIN enrollments e ON s.student_id = e.student_id
INNER JOIN courses c ON e.course_id = c.course_id;
-- LEFT JOIN: all students, even those not enrolled
SELECT s.name, c.course_name
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
LEFT JOIN courses c ON e.course_id = c.course_id;
-- Students with no enrollments show NULL for course_name
```

Self JOIN

```
-- Find employees and their managers (same table)
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

Chapter 6

Entity-Relationship (ER) Modeling

ER Model Components

Component	Symbol	Description	Example
Entity	Rectangle	A real-world object or concept	Student, Course, Order
Attribute	Oval	Property of an entity	name, email, gpa
Relationship	Diamond	Association between entities	enrolls_in, purchases
Primary Key	Underlined attribute	Uniquely identifies each entity	student_id
Cardinality	1, M, N on lines	How many of each entity participate	1:M, M:N

Cardinality Types

- **One-to-One (1:1)**: One person has one passport. Foreign key in either table.
- **One-to-Many (1:M)**: One department has many employees. FK in the 'many' side.
- **Many-to-Many (M:N)**: Students enroll in courses; courses have many students. Requires a **junction table** (e.g., enrollments) with FKs to both tables.

ER to Relational Mapping

(1) Each entity → a table. (2) Each attribute → a column. (3) Primary key → PRIMARY KEY constraint. (4) 1:M relationship → FK in the 'many' side table. (5) M:N relationship → junction table with two FKs. (6) 1:1 → FK in either table (preferably the one with total participation).

Chapter 7

Schema Design and Data Types

Common Data Types (PostgreSQL/Standard SQL)

Category	Types	Example
Integer	SMALLINT, INTEGER, BIGINT, SERIAL	user_id INTEGER
Decimal	NUMERIC(p,s), DECIMAL, REAL, DOUBLE	price NUMERIC(10,2)
String	CHAR(n), VARCHAR(n), TEXT	name VARCHAR(100)
Boolean	BOOLEAN	is_active BOOLEAN DEFAULT TRUE
Date/Time	DATE, TIME, TIMESTAMP, INTERVAL	created_at TIMESTAMP
Binary	BYTEA, BLOB	profile_pic BYTEA
JSON	JSON, JSONB (PostgreSQL)	metadata JSONB
UUID	UUID	id UUID DEFAULT gen_random_uuid()

Schema Design Best Practices

- Use **singular table names** (user, not users) — conventions vary, be consistent.
- Use **snake_case** for column names: first_name, not firstName.
- Every table should have a **primary key** (prefer surrogate keys like SERIAL/UUID).
- Add **created_at** and **updated_at** timestamps to every table.
- Use **NOT NULL** by default — allow NULL only when absence of data is meaningful.
- Use **appropriate data types** — don't store numbers as strings, dates as strings, etc.

Chapter 8

Keys, Constraints, and Referential Integrity

Types of Keys

Key	Definition	Example
Primary Key (PK)	Uniquely identifies each row; NOT NULL + UNIQUE	student_id
Foreign Key (FK)	References PK of another table; enforces relationships	dept_id REFERENCES departments(id)
Candidate Key	Column(s) that could serve as PK	email (unique, not null)
Composite Key	PK made of multiple columns	(student_id, course_id)
Surrogate Key	System-generated PK (no business meaning)	SERIAL, UUID
Natural Key	PK from real-world data	SSN, ISBN (risky if values change)

Constraints

```
CREATE TABLE orders (
    order_id     SERIAL PRIMARY KEY,
    customer_id  INTEGER NOT NULL REFERENCES customers(id)
                 ON DELETE CASCADE,       -- delete orders if customer deleted
    total        NUMERIC(10,2) CHECK (total >= 0),
    status       VARCHAR(20) DEFAULT 'pending'
                 CHECK (status IN ('pending','shipped','delivered','cancelled')),
    created_at   TIMESTAMP DEFAULT NOW(),
    UNIQUE (customer_id, created_at)     -- composite unique constraint
);
```

Referential Integrity Actions

Action	ON DELETE	ON UPDATE
CASCADE	Delete child rows	Update child FKs
SET NULL	Set child FK to NULL	Set child FK to NULL
SET DEFAULT	Set child FK to default	Set child FK to default
RESTRICT	Prevent delete if children exist	Prevent update if referenced
NO ACTION	Same as RESTRICT (deferred check)	Same as RESTRICT

Chapter 9

Subqueries and Views

Subqueries

```
-- Scalar subquery (returns one value)
SELECT name, gpa FROM students
WHERE gpa > (SELECT AVG(gpa) FROM students);
-- IN subquery (returns a list)
SELECT name FROM students
WHERE student_id IN (
    SELECT student_id FROM enrollments WHERE course_id = 101
);
-- EXISTS subquery (returns true/false)
SELECT name FROM students s
WHERE EXISTS (
    SELECT 1 FROM enrollments e
    WHERE e.student_id = s.student_id
);
-- Correlated subquery (references outer query)
SELECT name, gpa FROM students s
WHERE gpa > (SELECT AVG(gpa) FROM students
             WHERE department = s.department);
```

Views

A **view** is a saved query that acts like a virtual table. It does not store data — it executes the query each time. Benefits: simplify complex queries, provide security (expose only certain columns), and create a stable interface for applications.

```
CREATE VIEW honor_students AS
SELECT name, email, gpa FROM students WHERE gpa >= 3.5;
-- Use it like a table:
SELECT * FROM honor_students WHERE name LIKE 'A%';
-- Materialized view (stores results, must refresh):
CREATE MATERIALIZED VIEW monthly_sales AS
SELECT date_trunc('month', order_date) AS month, SUM(total)
FROM orders GROUP BY 1;
REFRESH MATERIALIZED VIEW monthly_sales; -- manual refresh
```

Chapter 10

Introduction to Normalization

Why Normalize?

Normalization eliminates redundancy and anomalies. Without it: **Insertion anomaly** — cannot add data without unrelated data (e.g., can't add a department without an employee). **Update anomaly** — changing one fact requires updating many rows. **Deletion anomaly** — deleting data accidentally removes unrelated data.

Normal Forms (Overview)

Form	Rule	Eliminates
1NF	Atomic values, no repeating groups	Multi-valued cells
2NF	1NF + no partial dependencies	Non-key depends on part of composite key
3NF	2NF + no transitive dependencies	Non-key depends on another non-key
BCNF	Every determinant is a candidate key	Remaining anomalies from 3NF

Most production databases target **3NF or BCNF**. Deeper normal forms (4NF, 5NF) exist but are rarely needed. Denormalization (intentionally breaking normalization for performance) is common in data warehouses and read-heavy workloads — covered in Part III.

PART II

INTERMEDIATE

For practitioners deepening their skills

Chapter 11

Normalization: 1NF through BCNF

Functional Dependencies

A **functional dependency** X → Y means: for any two rows, if they agree on X, they must agree on Y. Example: student_id → student_name (a student_id determines exactly one name). Understanding FDs is essential for normalization.

1NF: First Normal Form

A table is in 1NF if: (1) All columns contain atomic (indivisible) values. (2) No repeating groups or arrays. Violation: storing 'Math, Physics, Chemistry' in a single 'courses' column. Fix: separate rows, or a junction table.

2NF: Second Normal Form

A table is in 2NF if: it is in 1NF AND every non-key attribute depends on the **entire** primary key (no partial dependencies). This only matters for composite primary keys. If PK is (student_id, course_id) and student_name depends only on student_id, that is a partial dependency. Fix: move student_name to a students table.

3NF: Third Normal Form

A table is in 3NF if: it is in 2NF AND no non-key attribute depends on another non-key attribute (no transitive dependencies). If student_id → dept_id → dept_name, then dept_name transitively depends on student_id via dept_id. Fix: create a departments table.

BCNF: Boyce-Codd Normal Form

A table is in BCNF if: for every functional dependency X → Y, X is a superkey. BCNF is stricter than 3NF. Most tables in 3NF are also in BCNF. Exceptions arise with overlapping candidate keys.

Chapter 12

Indexing Fundamentals: B-Tree and Hash

Why Indexes?

Without an index, a query like WHERE email = 'alice@example.com' must scan every row (**full table scan**). With an index on email, the DB jumps directly to the matching row. Indexes trade write speed and storage for read speed.

B-Tree Index

The default index type in most databases. A **B-Tree** (balanced tree) maintains sorted data in a tree structure. Each node contains multiple keys and pointers. Leaf nodes link together for range scans. Complexity: O(log n) for lookup, insert, delete. Supports: equality (=), range (<, >, BETWEEN), ORDER BY, and prefix LIKE ('A%').

Hash Index

Uses a hash function to map keys to buckets. O(1) average lookup — faster than B-Tree for exact match. But: does not support range queries, ORDER BY, or partial matches. PostgreSQL supports hash indexes; MySQL/InnoDB does not (uses adaptive hash index internally).

Index Usage Guidelines

Do Index	Don't Index
Columns in WHERE clauses	Columns rarely used in queries
JOIN columns (foreign keys)	Low-cardinality columns (e.g., boolean)
ORDER BY / GROUP BY columns	Small tables (full scan is fine)
Columns in UNIQUE constraints	Frequently updated columns (high write cost)

```
-- Create index
CREATE INDEX idx_students_email ON students(email);
-- Composite index (leftmost prefix rule applies)
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date DESC);
-- Supports: WHERE customer_id = 5
-- Supports: WHERE customer_id = 5 AND order_date > '2024-01-01'
-- Does NOT support: WHERE order_date > '2024-01-01' (alone)
```

Chapter 13

Transactions and ACID Properties

What Is a Transaction?

A **transaction** is a group of operations that must succeed or fail as a unit. Classic example: bank transfer — debit from account A and credit to account B. If the debit succeeds but credit fails, money vanishes. Transactions prevent this.

ACID Properties

Property	Meaning	Mechanism
Atomicity	All or nothing — entire transaction succeeds or rolls back	WAL (Write-Ahead Log), undo log
Consistency	Database moves from one valid state to another	Constraints, triggers, application logic
Isolation	Concurrent transactions don't interfere with each other	Locks, MVCC, isolation levels
Durability	Committed data survives crashes	WAL flushed to disk before commit

```
-- Transaction example:
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
-- If both succeed:
COMMIT;
-- If anything fails:
ROLLBACK;  -- undo all changes
```

Chapter 14

Concurrency Control: Locks and MVCC

The Problem

Multiple transactions accessing the same data concurrently can cause: **Dirty read** (read uncommitted data), **Non-repeatable read** (same query returns different results within a transaction), **Phantom read** (new rows appear between queries), and **Lost update** (two transactions overwrite each other).

Locking

Shared lock (S): Multiple transactions can read simultaneously. **Exclusive lock (X)**: Only one transaction can write; blocks all others. **Two-Phase Locking (2PL)**: A transaction must acquire all locks before releasing any. Guarantees serializability but can cause deadlocks.

MVCC (Multi-Version Concurrency Control)

Used by PostgreSQL, Oracle, MySQL/InnoDB. Instead of blocking readers, MVCC keeps **multiple versions** of each row. Readers see a **snapshot** of the database at the start of their transaction. Writers create new versions. This means **readers never block writers, and writers never block readers**. Much better concurrency than pure locking.

Isolation Levels

Level	Dirty Read	Non-Repeatable	Phantom
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes (InnoDB: No)
SERIALIZABLE	No	No	No

PostgreSQL default: READ COMMITTED. MySQL/InnoDB default: REPEATABLE READ.

Chapter 15

SQL Advanced: Window Functions and CTEs

Window Functions

Window functions perform calculations across a set of rows **related to the current row** — without collapsing rows like GROUP BY. They use OVER() to define the window.

```
-- ROW_NUMBER: unique sequential number
SELECT name, department, salary,
  ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC)
  AS rank_in_dept
FROM employees;
-- Running total
SELECT order_date, amount,
  SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM orders;
-- Compare to previous row (LAG)
SELECT month, revenue,
  revenue - LAG(revenue) OVER (ORDER BY month) AS growth
FROM monthly_metrics;
```

Common Table Expressions (CTEs)

```
-- CTE for readability
WITH high_earners AS (
    SELECT * FROM employees WHERE salary > 100000
),
dept_counts AS (
    SELECT department, COUNT(*) AS cnt FROM high_earners
    GROUP BY department
)
SELECT * FROM dept_counts WHERE cnt > 5;
-- Recursive CTE (organizational hierarchy)
WITH RECURSIVE org_chart AS (
    SELECT id, name, manager_id, 1 AS level
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.id, e.name, e.manager_id, oc.level + 1
    FROM employees e JOIN org_chart oc ON e.manager_id = oc.id
)
SELECT * FROM org_chart ORDER BY level;
```

Chapter 16

Stored Procedures, Triggers, and Functions

Stored Procedures

A **stored procedure** is a precompiled SQL program stored in the database. Benefits: reduce network round-trips, encapsulate business logic, reusable across applications.

```
-- PostgreSQL (PL/pgSQL)
CREATE OR REPLACE FUNCTION transfer_funds(
    from_id INT, to_id INT, amount NUMERIC
) RETURNS VOID AS $$
BEGIN
    UPDATE accounts SET balance = balance - amount WHERE id = from_id;
    UPDATE accounts SET balance = balance + amount WHERE id = to_id;
    IF NOT FOUND THEN RAISE EXCEPTION 'Account not found'; END IF;
END;
$$ LANGUAGE plpgsql;
-- Usage: SELECT transfer_funds(1, 2, 500.00);
```

Triggers

A **trigger** is code that fires automatically on INSERT, UPDATE, or DELETE. Use cases: audit logging, enforcing complex constraints, updating computed columns.

```
CREATE TRIGGER audit_update
AFTER UPDATE ON employees
FOR EACH ROW
EXECUTE FUNCTION log_employee_change();  -- calls a function
```

Chapter 17

Query Execution Plans and EXPLAIN

How the DB Executes a Query

(1) **Parse**: Check syntax, validate tables/columns. (2) **Plan**: Query optimizer generates execution plans and picks the cheapest. (3) **Execute**: Run the chosen plan. The optimizer considers: available indexes, table statistics (row counts, data distribution), join order, and access methods.

Reading EXPLAIN

```
EXPLAIN ANALYZE SELECT * FROM orders
WHERE customer_id = 42 AND total > 100;
-- Example output (PostgreSQL):
-- Index Scan using idx_orders_customer on orders
--   (cost=0.42..8.44 rows=1 width=48)
--   (actual time=0.023..0.025 rows=1 loops=1)
--   Index Cond: (customer_id = 42)
--   Filter: (total > 100)
--   Planning Time: 0.1 ms
--   Execution Time: 0.05 ms
```

Common Scan Types

Scan	Meaning	Performance
Seq Scan	Full table scan — reads every row	Slow for large tables
Index Scan	Uses index to find rows, then fetches from table	Fast for selective queries
Index Only Scan	All needed data is in the index	Fastest — no table access
Bitmap Index Scan	Scan index, build bitmap, then fetch	Good for medium selectivity

Chapter 18

Database Security and Access Control

Authentication and Authorization

Authentication: Who are you? (password, certificate, LDAP). **Authorization**: What can you do? (GRANT/REVOKE permissions on tables, schemas, functions).

```
-- Create a read-only role
CREATE ROLE analyst LOGIN PASSWORD 'secure_pass';
GRANT CONNECT ON DATABASE mydb TO analyst;
GRANT USAGE ON SCHEMA public TO analyst;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyst;
-- Row-Level Security (PostgreSQL)
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY user_orders ON orders
    USING (user_id = current_user_id());  -- users see only their rows
```

SQL Injection Prevention

NEVER concatenate user input into SQL queries. Always use parameterized queries / prepared statements.

```
-- VULNERABLE (never do this):
query = "SELECT * FROM users WHERE name = '" + user_input + "'"
-- SAFE (parameterized):
cursor.execute("SELECT * FROM users WHERE name = %s", (user_input,))
```

Chapter 19

Backup, Recovery, and WAL

Write-Ahead Logging (WAL)

Before any change is written to data files, it is first recorded in the **WAL** (Write-Ahead Log). On crash, the DB replays the WAL to recover committed transactions and undo uncommitted ones. This guarantees **durability** without flushing every data page to disk immediately.

Backup Strategies

Type	Method	Pros	Cons
Logical	pg_dump, mysqldump	Portable, human-readable	Slow for large DBs
Physical	Copy data files (pg_basebackup)	Fast, complete	Version-specific
Incremental	WAL archiving + base backup	Space-efficient, point-in-time	More complex setup
Snapshot	Filesystem/cloud snapshot	Instant, consistent	Requires FS support

Point-in-Time Recovery (PITR)

Combine a base backup with archived WAL files to restore the database to any specific point in time. Critical for recovering from accidental data deletion: restore to 1 minute before the DELETE was executed.

Chapter 20

Introduction to NoSQL

Why NoSQL?

Relational databases excel at structured data with complex relationships. But some workloads need: **flexible schemas** (documents change shape over time), **horizontal scalability** (spread across many servers), **high write throughput** (millions of writes/sec), or **low-latency reads** (sub-millisecond). NoSQL databases sacrifice some RDBMS guarantees (often full ACID) for these benefits.

NoSQL Categories

Type	Data Model	Example	Trade-off
Document	JSON documents	MongoDB	Flexible schema, no JOINs
Key-Value	Key → blob	Redis, DynamoDB	Simple, fast, no queries on values
Column-Family	Row key → column families	Cassandra, HBase	Write-optimized, eventual consistency
Graph	Nodes + edges	Neo4j	Complex relationships, smaller scale

SQL vs NoSQL: When to Choose

Choose SQL When	Choose NoSQL When
Data is structured with clear relationships	Schema evolves frequently
ACID transactions are critical	Horizontal scalability is essential
Complex queries and JOINs are needed	Write throughput exceeds single-server capacity
Data integrity is paramount	Low-latency reads at massive scale

PART III

ADVANCED

For those seeking mastery of database internals

Chapter 21

Query Optimization Internals

How the Optimizer Works

The query optimizer transforms a SQL query into the most efficient execution plan. Steps: (1) **Parse** SQL into an AST. (2) **Rewrite** (apply rules: predicate pushdown, view expansion). (3) **Plan enumeration** (generate candidate plans). (4) **Cost estimation** (estimate I/O, CPU, memory cost using table statistics). (5) **Select cheapest plan**.

Key Optimization Techniques

- **Predicate pushdown**: Apply WHERE filters as early as possible (before JOINs).
- **Join reordering**: Small table first, large table second. Reduces intermediate result size.
- **Index selection**: Choose between seq scan, index scan, index-only scan based on selectivity.
- **Join algorithms**: Nested Loop (small tables), Hash Join (equality, large tables), Merge Join (pre-sorted data).
- **Materialization**: Compute subquery once and reuse (vs. re-execute per row).

Statistics

The optimizer relies on **statistics**: row counts, column cardinality (distinct values), histograms (data distribution), correlation. PostgreSQL: ANALYZE command updates stats. Stale stats → bad plans → slow queries. Run ANALYZE regularly or enable autovacuum (which does this automatically).

Chapter 22

Storage Engines: InnoDB, RocksDB, WiredTiger

Storage Engine Comparison

Engine	Used By	Structure	Strength
InnoDB	MySQL	B+Tree, clustered index	ACID, row-level locking, mature
RocksDB	CockroachDB, TiDB	LSM-Tree	High write throughput, compression
WiredTiger	MongoDB	B-Tree + LSM hybrid	Compression, concurrency
PostgreSQL heap	PostgreSQL	Heap + B-Tree indexes	MVCC, extensible, feature-rich

B+Tree vs LSM-Tree

Aspect	B+Tree	LSM-Tree (Log-Structured Merge)
Write pattern	In-place update	Append-only (write to memtable → SSTable)
Read speed	Fast (direct index lookup)	Slower (check memtable + multiple SSTables)
Write speed	Slower (random I/O for in-place update)	Fast (sequential writes)
Space amplification	Low (one copy of data)	Higher (multiple copies until compaction)
Compaction	Not needed	Required (merge SSTables in background)
Best for	Read-heavy OLTP	Write-heavy workloads

Chapter 23

Advanced Indexing: GiST, GIN, Bloom, Bitmap

Beyond B-Tree

Index Type	Use Case	Database
B-Tree	Default: equality, range, sorting	All RDBMS
Hash	Equality only (faster than B-Tree for =)	PostgreSQL, internal in InnoDB
GiST	Geometric data, full-text search, range types	PostgreSQL
GIN (Inverted)	Full-text search, JSONB, arrays	PostgreSQL
BRIN (Block Range)	Naturally ordered data (timestamps)	PostgreSQL
Bitmap	Low-cardinality columns, data warehousing	Oracle, PostgreSQL (internally)
Bloom Filter	Multi-column equality when B-Tree would need many indexes	PostgreSQL (extension)
Partial Index	Index only rows matching a condition	PostgreSQL, SQLite

```
-- GIN index for JSONB queries
CREATE INDEX idx_metadata ON products USING gin(metadata);
SELECT * FROM products WHERE metadata @> '{"color": "red"}';
-- BRIN index for time-series (huge tables, tiny index)
CREATE INDEX idx_events_time ON events USING brin(created_at);
-- Partial index (only index active users)
CREATE INDEX idx_active_users ON users(email) WHERE is_active = true;
```

Chapter 24

Isolation Levels and Anomalies Deep Dive

Anomalies in Detail

- **Dirty Read**: T1 reads data written by T2 before T2 commits. If T2 rolls back, T1 read data that never existed.
- **Non-Repeatable Read**: T1 reads a row, T2 modifies it and commits, T1 reads again and gets different data.
- **Phantom Read**: T1 queries rows matching a condition, T2 inserts a new matching row, T1 re-queries and sees an extra row.
- **Write Skew**: Two transactions read overlapping data, make decisions based on it, and write non-conflicting data — but the combined result violates a constraint. Only prevented by SERIALIZABLE.

Snapshot Isolation

Both PostgreSQL's REPEATABLE READ and MySQL/InnoDB's REPEATABLE READ use **snapshot isolation**: each transaction sees a consistent snapshot from its start. This prevents dirty reads, non-repeatable reads, and phantoms. However, it allows **write skew**. PostgreSQL's SERIALIZABLE level uses **Serializable Snapshot Isolation (SSI)** — it detects and aborts transactions that would cause write skew, with minimal performance overhead.

Chapter 25

Distributed Databases and CAP Theorem

CAP Theorem

In a distributed system, you can have at most 2 of 3: **Consistency** (every read gets the latest write), **Availability** (every request gets a response), **Partition tolerance** (system works despite network splits). Since partitions are inevitable, the real choice is CP (consistent but may be unavailable) vs AP (available but may return stale data).

Consistency Models

Model	Guarantee	Example
Strong consistency	Reads always see latest write	Spanner, CockroachDB
Linearizability	Operations appear to execute in real-time order	etcd, ZooKeeper
Eventual consistency	Reads may be stale but will converge	DynamoDB, Cassandra
Causal consistency	Causally related operations seen in order	MongoDB (causal sessions)

Chapter 26

Replication: Single-Leader, Multi-Leader, Leaderless

Replication Architectures

Architecture	How It Works	Pros	Cons
Single-Leader	One writable primary, read-only replicas	Simple, no conflicts	Primary is bottleneck/SPOF
Multi-Leader	Multiple writable nodes	Write availability, low latency	Conflict resolution complex
Leaderless	Any node accepts reads and writes	High availability	Requires quorum, eventual consistency

Replication Lag

In async replication, replicas lag behind the primary. This causes: **read-after-write inconsistency** (write to primary, read from stale replica), **monotonic read violations** (time seems to go backward). Solutions: read-from-primary for recent writes, causal consistency, or synchronous replication (at cost of latency).

Chapter 27

Partitioning and Sharding

Why Partition?

When a single server cannot handle the data volume or query load, split data across multiple nodes. **Partitioning** = splitting within one DB instance (e.g., PostgreSQL table partitioning). **Sharding** = splitting across multiple DB instances.

Partitioning Strategies

Strategy	Method	Pros	Cons
Range	Split by key range (e.g., dates)	Good for range queries	Hot spots if access is uneven
Hash	Hash(key) mod N shards	Even distribution	Range queries span all shards
List	Explicit value mapping	Control over placement	Manual management
Composite	Range on one key, hash on another	Balances both needs	Complex

Cross-Shard Queries

The biggest challenge: queries that span multiple shards (scatter-gather) are slow. Transactions across shards require distributed coordination (2PC). Design your sharding key so that most queries hit a single shard. Common keys: tenant_id (multi-tenant SaaS), user_id (user-centric apps), region (geographic data).

Chapter 28

Document Databases: MongoDB In Depth

MongoDB Data Model

MongoDB stores data as **BSON documents** (binary JSON) in **collections** (like tables). Documents can have nested objects and arrays — no fixed schema. This makes it easy to evolve the data model without migrations.

```
// MongoDB document example:
{
  _id: ObjectId("65a1b2c3d4e5f6a7b8c9d0e1"),
  name: "Alice Smith",
  email: "alice@example.com",
  addresses: [
    { type: "home", city: "New York", zip: "10001" },
    { type: "work", city: "Boston", zip: "02101" }
  ],
  orders: [
    { item: "Laptop", price: 1299.99, date: ISODate("2024-01-15") }
  ]
}
```

When to Embed vs Reference

Embed (denormalize)	Reference (normalize)
Data is always accessed together	Data is accessed independently
One-to-few relationship	One-to-many or many-to-many
Data rarely changes	Data changes frequently
Document stays under 16 MB	Large or unbounded sub-documents

Chapter 29

Key-Value and Column-Family: Redis, Cassandra

Redis

Redis is an in-memory key-value store. Sub-millisecond latency. Data structures: strings, lists, sets, sorted sets, hashes, streams, bitmaps. Use cases: caching, session storage, leaderboards, rate limiting, pub/sub. Redis can persist to disk (RDB snapshots, AOF log) but is primarily an in-memory system. Redis Cluster provides horizontal scaling via hash slot partitioning (16384 slots).

Apache Cassandra

Cassandra is a wide-column (column-family) database designed for massive write throughput and high availability. Key features: **Masterless** (no single point of failure — all nodes are equal), **tunable consistency** (choose consistency level per query: ONE, QUORUM, ALL), **linear scalability** (add nodes, throughput scales linearly). Data model: partition key determines which node stores the data; clustering columns determine sort order within a partition.

Cassandra Data Modeling

Unlike RDBMS, Cassandra requires **query-driven design**: design your tables around the queries you need to run, not around entities. Each query pattern gets its own table. Denormalization is expected and encouraged. JOINs are not supported — all data for a query must be in one table.

Chapter 30

Graph Databases and Neo4j

When to Use a Graph Database

When relationships are the primary concern: social networks (friends of friends), recommendation engines, fraud detection, knowledge graphs, network topology. Relational databases can model graphs with join tables, but performance degrades exponentially as traversal depth increases. Graph databases are optimized for traversals — O(1) per relationship hop.

Neo4j and Cypher

```
// Create nodes and relationships
CREATE (alice:Person {name: 'Alice', age: 30})
CREATE (bob:Person {name: 'Bob', age: 25})
CREATE (alice)-[:FRIENDS_WITH {since: 2020}]->(bob);
// Find friends of friends
MATCH (p:Person {name: 'Alice'})-[:FRIENDS_WITH*2]->(fof)
RETURN fof.name;
// Shortest path
MATCH path = shortestPath(
  (a:Person {name:'Alice'})-[:FRIENDS_WITH*]-(b:Person {name:'Eve'})
) RETURN path;
```

PART IV

PRO / EXPERT

Staff-level judgment and craft

Chapter 31

PostgreSQL Internals Deep Dive

Process Architecture

PostgreSQL uses a **process-per-connection** model. The **postmaster** process listens for connections and forks a backend process for each client. Background processes: **autovacuum** (reclaim dead tuples from MVCC), **WAL writer** (flush WAL to disk), **checkpointer** (write dirty pages to disk), **bgwriter** (pre-flush dirty pages), **stats collector** (gather table/index statistics).

MVCC Implementation

Each row version has **xmin** (transaction that created it) and **xmax** (transaction that deleted/updated it). A transaction sees a row if: xmin is committed and < current txid, AND (xmax is empty OR xmax is > current txid or aborted). Dead tuples (invisible to all active transactions) are cleaned by **VACUUM**. Without vacuuming: table bloat, index bloat, transaction ID wraparound.

Query Processing Pipeline

```
// PostgreSQL query pipeline:
SQL text -> Parser (AST) -> Analyzer (semantic check)
  -> Rewriter (view expansion, rules)
  -> Planner/Optimizer (cost-based, generates plan tree)
  -> Executor (runs the plan, returns results)
```

Chapter 32

MySQL/InnoDB Architecture

InnoDB Architecture

InnoDB is MySQL's default storage engine. Key components: **Buffer Pool** (caches data and index pages in memory — the most important tuning parameter), **Redo Log** (WAL for crash recovery), **Undo Log** (for MVCC and rollbacks), **Change Buffer** (caches changes to secondary indexes), **Adaptive Hash Index** (automatic hash index on frequently accessed pages).

Clustered Index

InnoDB stores data in the **primary key** order (clustered index). The leaf nodes of the primary key B+Tree contain the actual row data. Secondary indexes store the primary key value (not a row pointer). This means secondary index lookups require two B+Tree traversals: secondary index → find PK → primary index → find row. Implications: keep primary keys small (UUID = 16 bytes vs INT = 4 bytes — affects every secondary index).

InnoDB vs PostgreSQL

Aspect	InnoDB (MySQL)	PostgreSQL
Storage	Clustered index (data in PK order)	Heap (data in insertion order)
MVCC	Undo log (old versions in undo space)	In-place (old versions in main table)
Vacuuming	Purge thread (automatic)	VACUUM required (autovacuum)
Full-text search	Basic	Advanced (tsvector, GIN)
JSON support	JSON type	JSONB (binary, indexable)
Replication	Binlog-based	WAL-based (logical/physical)

Chapter 33

Consensus Protocols: Paxos, Raft, 2PC

Two-Phase Commit (2PC)

Used for distributed transactions. Phase 1 (**Prepare**): Coordinator asks all participants 'Can you commit?'. Each participant writes to its log and votes YES/NO. Phase 2 (**Commit**): If all vote YES, coordinator sends COMMIT; otherwise ABORT. Problem: if coordinator crashes after prepare but before commit, participants are **blocked** — they cannot decide alone. Solution: 3PC or consensus protocols.

Raft

Raft is an understandable consensus protocol (used by etcd, CockroachDB, TiDB). Key concepts: **Leader election** (one node is leader, others are followers), **Log replication** (leader replicates log entries to followers), **Safety** (once a log entry is committed by a majority, it is permanent). Leader sends heartbeats; if followers don't hear from leader, they start an election. A log entry is committed when a majority of nodes have it.

Paxos

The original consensus algorithm (Lamport, 1989). More general but harder to understand than Raft. Multi-Paxos is used in production systems (Google Spanner, Chubby). The core idea: a proposer proposes a value, acceptors vote, and a value is chosen when accepted by a majority. Multiple rounds may be needed.

Chapter 34

NewSQL: CockroachDB, TiDB, Spanner

What Is NewSQL?

NewSQL databases combine the scalability of NoSQL with the ACID guarantees and SQL interface of traditional RDBMS. They provide **distributed SQL** — horizontal scaling without sacrificing transactions.

Comparison

Database	Architecture	Consensus	Storage
Google Spanner	Globally distributed, TrueTime (GPS+atomic clocks)	Paxos	Custom (Colossus)
CockroachDB	Spanner-inspired, open source	Raft	RocksDB (LSM)
TiDB	MySQL-compatible, HTAP	Raft	TiKV (RocksDB)
YugabyteDB	PostgreSQL-compatible, distributed	Raft	DocDB

When to Use NewSQL

When you need: global distribution with strong consistency, horizontal write scaling, ACID transactions across shards, and SQL compatibility. Trade-offs: higher latency than single-node RDBMS (network round-trips for consensus), operational complexity, and cost. For most applications, a well-tuned single-node PostgreSQL or MySQL is simpler and sufficient.

Chapter 35

Time-Series Databases: InfluxDB, TimescaleDB

Time-Series Data Characteristics

Time-series data is timestamped, append-mostly, high-volume, and often queried by time range. Examples: server metrics, IoT sensor data, financial ticks, application logs. Regular RDBMS can handle time-series but struggle at scale because: B-Tree indexes become bottlenecks for high-volume inserts, and time-range queries require scanning large portions of the index.

TimescaleDB

A PostgreSQL extension for time-series. Automatically partitions data into **chunks** by time (e.g., one chunk per day). Each chunk is a regular PostgreSQL table. Benefits: full SQL, JOINs with relational data, all PostgreSQL features. Compression: 90%+ reduction. Continuous aggregates: pre-computed rollups. Retention policies: auto-drop old chunks.

InfluxDB

Purpose-built time-series DB. Uses its own query language (Flux) and a custom storage engine optimized for time-series writes. Key concepts: **measurement** (like a table), **tags** (indexed metadata), **fields** (values), **timestamp**. Very fast ingestion but limited query flexibility compared to SQL.

Chapter 36

Vector Databases and Embeddings

What Are Vector Databases?

AI/ML models (GPT, CLIP, BERT) convert text, images, and audio into **embeddings** — high-dimensional vectors (e.g., 1536 floats). Vector databases store these embeddings and support **similarity search**: given a query vector, find the K most similar vectors. This powers: semantic search, recommendation engines, RAG (Retrieval-Augmented Generation) for LLMs, image search, and anomaly detection.

Similarity Search Algorithms

Algorithm	Type	Speed	Accuracy
Flat (brute force)	Exact	Slow O(n)	Perfect
IVF (Inverted File Index)	Approximate	Fast	Good (with enough probes)
HNSW (Hierarchical NSW)	Approximate	Very fast	Excellent
PQ (Product Quantization)	Approximate + compressed	Fast, low memory	Good

pgvector

pgvector is a PostgreSQL extension that adds vector similarity search to PostgreSQL. You can store embeddings alongside relational data and query both with SQL. This avoids a separate vector database for many use cases.

```
-- pgvector example:
CREATE EXTENSION vector;
CREATE TABLE documents (
    id SERIAL PRIMARY KEY,
    content TEXT,
    embedding vector(1536)  -- OpenAI embedding dimension
);
CREATE INDEX ON documents USING hnsw (embedding vector_cosine_ops);
-- Find 5 most similar documents:
SELECT content, 1 - (embedding <=> query_vec) AS similarity
FROM documents ORDER BY embedding <=> query_vec LIMIT 5;
```

Chapter 37

Data Warehousing: Star Schema, OLAP, Snowflake

OLTP vs OLAP

Aspect	OLTP	OLAP
Purpose	Transaction processing	Analytical queries
Queries	Simple, many small transactions	Complex, few large aggregations
Data model	Normalized (3NF)	Denormalized (star/snowflake)
Users	Application users (thousands)	Analysts, dashboards (few)
Examples	PostgreSQL, MySQL	Snowflake, BigQuery, Redshift

Star Schema

The star schema has a central **fact table** (transactions, events — contains measures and FK pointers) surrounded by **dimension tables** (descriptive attributes). Example: fact_sales (date_id, product_id, store_id, quantity, revenue) joins to dim_date, dim_product, dim_store. Simple, fast for aggregation queries.

Columnar Storage

OLAP databases store data **column-by-column** instead of row-by-row. Benefits: (1) Only read columns needed for the query. (2) Same-type data compresses much better (10-100x). (3) Vectorized processing (SIMD on columns). This is why analytical queries on columnar DBs are 10-100x faster than on row stores.

Chapter 38

Stream Processing and Change Data Capture

Change Data Capture (CDC)

CDC captures changes (inserts, updates, deletes) from a database's transaction log and streams them to other systems. This enables: real-time data pipelines, cache invalidation, search index updates, and event-driven architectures. Tools: **Debezium** (open source, reads PostgreSQL WAL / MySQL binlog), AWS DMS, Fivetran.

Stream Processing

Apache Kafka is the standard message broker for stream processing. It provides durable, ordered, replayable event logs. Kafka Connect integrates with databases (via CDC). Stream processors (**Kafka Streams**, **Apache Flink**, **ksqlDB**) consume events, transform them, and produce results in real-time.

Event Sourcing

Instead of storing current state (UPDATE), store every event that led to the current state (APPEND). Benefits: complete audit trail, temporal queries, rebuild state at any point in time. Challenge: increased storage, complex querying. Often combined with CQRS (Command Query Responsibility Segregation).

Chapter 39

Database Reliability Engineering

SLOs and Error Budgets

Define **SLOs** (Service Level Objectives) for your database: availability (99.99% = 52 min downtime/year), latency (p99 < 50 ms), durability (zero data loss). The **error budget** is the allowed downtime — spend it on deployments, maintenance, and experiments.

Failure Modes

Failure	Impact	Mitigation
Disk failure	Data loss if no replication	RAID, replication, backups
Primary crash	Write unavailability	Automatic failover (Patroni, RDS Multi-AZ)
Replication lag	Stale reads	Synchronous replication, read-from-primary
Table bloat	Slow queries, wasted space	Autovacuum tuning (PostgreSQL)
Connection exhaustion	New connections rejected	Connection pooling (PgBouncer)
Long-running queries	Lock contention, OOM	Statement timeout, pg_stat_activity monitoring

Connection Pooling

Opening a database connection is expensive (~5-20 ms). A **connection pooler** (PgBouncer, ProxySQL) maintains a pool of open connections and multiplexes client requests. Modes: **session** (one pool connection per client session — safest), **transaction** (release connection between transactions — most efficient), **statement** (release after each statement — most aggressive).

Chapter 40

Database Design Patterns and Trade-offs

Fundamental Trade-offs

Trade-off	Option A	Option B	Guidance
Normalize vs Denormalize	3NF: no redundancy, small writes	Denorm: fast reads, redundant data	OLTP: normalize. OLAP: denormalize
SQL vs NoSQL	ACID, SQL, mature ecosystem	Scale, flexible schema, speed	Default to SQL unless specific need
Consistency vs Availability	Strong consistency (CP)	High availability (AP)	CP for financial; AP for social feeds
Embed vs Reference	Single document, fast read	Separate collections, normalized	Embed if always accessed together
Single DB vs Microservice DBs	Simple, consistent	Independent scaling, isolation	Start single; split when team grows

Design Patterns

- **Soft deletes**: Add deleted_at column instead of DELETE. Enables undo and audit trail.
- **Temporal tables**: Store valid_from/valid_to for historical queries (system versioning in SQL:2011).
- **Polymorphic associations**: entity_type + entity_id columns to reference different tables.
- **Materialized views**: Pre-compute expensive queries. Refresh on schedule or via triggers.
- **Database per tenant**: Maximum isolation for multi-tenant SaaS. Vs. shared schema with tenant_id column.
- **CQRS**: Separate read models (optimized views) from write models (normalized tables).

Closing Thought

Databases are the foundation of nearly every application. Choosing the right database, designing a good schema, writing efficient queries, and understanding the internals are among the most valuable skills in software engineering. The best database is the one your team understands, operates confidently, and that meets your workload requirements. Start simple (PostgreSQL covers 90%+ of use cases), measure, and evolve.

QUICK REFERENCE

Decision guides, cheat sheets, and at-a-glance comparisons

Quick Reference

DBMS Cheat Sheet

SQL Query Execution Order

```
FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT
-- Written as:  SELECT ... FROM ... WHERE ... GROUP BY ...
-- Executed as: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY
```

JOIN Quick Reference

```
INNER JOIN  = matching rows in both tables
LEFT JOIN   = all left + matching right (NULL if no match)
RIGHT JOIN  = all right + matching left
FULL JOIN   = all from both (NULL where no match)
CROSS JOIN  = every combination (Cartesian product)
```

Normalization Rules

```
1NF: Atomic values, no repeating groups
2NF: 1NF + no partial dependencies (on part of composite PK)
3NF: 2NF + no transitive dependencies (non-key -> non-key)
BCNF: Every determinant is a candidate key
```

ACID Properties

```
Atomicity:    All or nothing (transaction succeeds or rolls back)
Consistency:  DB moves from valid state to valid state
Isolation:    Concurrent txns don't interfere
Durability:   Committed data survives crashes (WAL)
```

Isolation Levels

Level	Dirty Read	Non-Repeat	Phantom	Write Skew
READ UNCOMMITTED	Yes	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes	Yes
REPEATABLE READ	No	No	PG:No / Std:Yes	Yes
SERIALIZABLE	No	No	No	No

Index Type Quick Reference

```
B-Tree:    Default. Equality, range, sorting, LIKE 'prefix%'
Hash:      Equality only. Faster than B-Tree for =
GIN:       Full-text, JSONB, arrays (inverted index)
GiST:      Geometric, range types, nearest-neighbor
BRIN:      Huge tables with natural ordering (timestamps)
Partial:   Index only rows matching a condition
```

Common PostgreSQL Commands

```
\dt             -- list tables
\d tablename    -- describe table structure
```

```
\di              -- list indexes
\x               -- toggle expanded display
EXPLAIN ANALYZE   -- show actual execution plan
VACUUM ANALYZE    -- reclaim space + update statistics
pg_stat_activity  -- see active queries
pg_stat_user_tables -- table-level statistics
```